Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

Smart University Timetable Management System

Team Name:	*Catalyst*
Team member 1 (Team Lead)	*Tripathi, Shreya-2510011705* *shreyatripathi1692@gmail.com* 
Team member 2	*Kanupriya-2510016460* *kanupriya1220@gmail.com* 
Team member 3	*Nautiyal, Medha-2510012378* *medhanautiyal06@gmail.com* 
Team member 4	*Abhimanyue Kush, Avika-2510012834* *kush.avikaa@gmail.com* 

PHASE-II PROJECT PROGRESS DESCRIPTION (35 pts)

Project Abstract (2 pts)

(Brief restatement of your project's main goal. Max 300 words).

Manual college timetables are difficult to manage and generate and have a high chance to clash with each other . The main goal of Catalyst is to ensure conflict free timetable generation and synchronise all the changes made after creation. The C++ scheduling engine uses backtracking to explore possible assignments and the Minimum Remaining Values (MRV) to schedule the most constrained sessions allocated first. Hard constraints prevent a teacher, room, or section from being assigned to overlapping classes. Because sections may share faculty and rooms, they are scheduled together against a common set of resource bookings. Soft constraints can help improve schedule quality by balancing classes across the week and reducing gaps in students' days.
Administrators creating the schedules ,faculty needing accurate teaching timing and reminder before the lecture are the people directly effected by Catalyst .

Updated Project Approach and Architecture (2 pts)

(Describe your current approach, including system design, communication protocols, libraries used, etc. Max 300 words).

Catalyst is a desktop application in C++ (Qt) with MySQL database .
It consists of three layers Qt Widgets Interface , c++ backend engine and MySQL database .

Backend (Core Engine):
Timetabling is a modeled as a constraint Satisfaction problem. Time table is generated using Backtracking algorithm along with MRV heuristic (Means placing the most constrained classes are allocated first)
-Hard Constraints (Constrains that must be satisfied)
1.No teacher , Room or section double-booked
2.Room capacity and Type must match
3.Each Subject gets its exact weekly sessions
-Soft Constraints (Score based) Even Distribution , fewer Gaps , best -fit Room
Three in-memory hash sets are used to conflict checks and handle teachers allocation in multiple sections
Change Requests by the faculty is checked against only the affected slot , then applied or rejected with a reason . The full timetable is not regenerated .

Database :
MySQL with 10 tables, designed by us: Teacher, Room, Subject, Course, Section, three link tables, TimeSlot and TimetableEntry.

Frontend:
A login dialog opens a role Based window : Super Admin (Full Access) , Admin , Faculty and Student
One reusable template grid is filtered per user .
It also contains a request change dialog and a conflict dialog (that explains reason behind rejected changes)

Tasks Completed (7 pts)

(Describe the main tasks that have been assigned and already completed. Max 250 words).

Task Completed	Team Member
- Designed and implemented the MySQL database for the timetable generation system, including table structures, relationships and data validation, while contributing to the development of the main application interface.	Shreya Tripathi
- Developed the main application interface, featuring the timetable overview and admin management screen, while contributing to the database design and integration requirements.	Medha Nautiyal
- Developed the faculty interface to display individual teachers' schedules and notification popup for faculty updates and alerts.	Kanupriya
- Prepared the project presentation (PPT). - Conducted testing to verify the system's functionality.	Avika

Challenges/Roadblocks (7 pts)

(Describe the challenges that you have faced or are facing so far and how you plan to solve them. Max 300 words).

1. Choosing a feasible project idea and the technologies under C++ that would be required for the implementation of the project and Finalizing the algorithm that decides how the backend would assign the classes.
Plan - using Qt -widget in C++ and backtracking algorithm and (MRV) heuristic for the backend.

2. Designing a clear and responsive interface that displays ,faculty , room ,and sections without becoming crowded ,clipped when resized.
Plan - Use Qt layouts and responsive size policies, organize information into clear sections, and test the dashboard at different window sizes.

3. Creating a timetable that remains balanced and handling later changes without introducing new conflicts.
Plan- First, enforce all hard constraints. Then use soft-constraint scores to reduce uneven days and student gaps. For each requested change, check the proposed slot and suggest valid alternatives if it causes a conflict.

4. Designing a database schema that correctly represents the relationships between faculty, courses, sections, rooms, sessions, and time slots, then connecting it reliably to the C++ application.
Plan - Design and review the schema, keys, and relationships first. Then test the C++ database connection with a small read/write operation before integrating the full application.

5. Protecting data and handling failures: Incorrect credentials, lost connections, or failed writes can interrupt the system.
Plan: Keep credentials out of source code, report errors clearly, and avoid showing a change as saved until the database confirms it.

Tasks Pending (7 pts)

(Describe the main tasks that you still need to complete. Max 250 words).

Task Pending	Team Member (to complete the task)
- Integrating the backend with the frontend and MySQL database. - Developing the core backend engine for timetable generation and management.	Shreya Tripathi
- Integrating the frontend with the backend. - Contributing to the development of the core back-end engine.	Medha Nautiyal
- Refining and improving the frontend interface for better usability, consistency, and user experience.	Kanupriya
- Designing and implementing the scheduling constraints and Minimum Remaining Values (MRV) heuristic for efficient timetable generation.	Avika

Project Outcome/Deliverables (2 pts)

(Describe what are the key outcomes / deliverables of the project. Max 200 words).

Automated Timetable generation : The system generates college timetables using constraint based backtracking and MRV heuristic , avoiding any scheduling conflicts .
Centralized Database : A MySQL database to manage teachers , rooms , subjects , time slots and timetable data entries .
Role Based Desktop Interface : Have separate interface for Super Admin , Admin , Faculty and students .
Faculty Notifications : Notifies the faculty 5 mins prior regarding their scheduled lecture .
Optimized Scheduling : Uses soft constraints to create a timetable having evenly distributed classes , minimized gapes , adequate breaks and suitable room allocation .
Maintainable Software Architecture: A modular C++ application using Qt Widgets, Qt SQL, MySQL, and CMake.

Progress Overview (2 pts)

(Summarize how much of the project is done, what's behind schedule, what's ahead of schedule. Max 200 words.)

The project has completed its initial research ,requirements gathering and architecture planning . A basic Qt 6 Widgets interface has been created and successfully built . The project also contains sample course, faculty, and timetable data for use during development.
The backend implementation using back-tracking , MRV , hard and soft constraints checks are in progress.
Next priority - Implement and test timetable generation, then connect the interface to stored data

Codebase Information (2 pts)

(Repository link, branch, and information about important commits.)

Repository:https://github.com/medhanautiyal06-crypto/catalyst.git

Branch: Main Branch

Current Status: Initial development is in progress, with database design and implementation completed and frontend development and backend integration underway.

Testing and Validation Status (2 pts)

(Provide information about any tests conducted)

Test Type	**Status (Pass/Fail)**	**Notes**
1.Database Schema and Relationships	Pass	Verified table structures, keys, and relationships.
2.Constraint Validation	Pass	Confirmed invalid data is rejected by database constraints.
3. Data Validation	Pass	Checked sample data, teacher workloads, section hours, and room capacity using SQL queries.
4.Backend and Timetable Generation	Pending	Backend engine is yet to be developed.
5. Frontend–Backend Integration	Pending	Integration testing will be conducted after backend development.
6.Change Request and Conflict Testing	Pending	To be tested after implementing the scheduling and conflict-checking logic.

Deliverables Progress (2 pts)

(Summarize the current status of all key project deliverables mentioned earlier. Indicate whether each deliverable is completed, in progress, or pending.)

Background research: Completed
Requirements gathering: Completed
System architecture plan: Completed
Qt 6 Widgets interface prototype: Completed
Sample course, faculty, and timetable data: Completed
Database schema and integration: In progress
C++ timetable-generation engine: In progress
Backtracking and MRV implementation: In progress
Hard-constraint conflict checking: In progress
Soft-constraint schedule optimization: Pending
Timetable-change validation and alternative-slot suggestions: Pending
Real-time synchronization and notifications: Pending